

Exercise 4

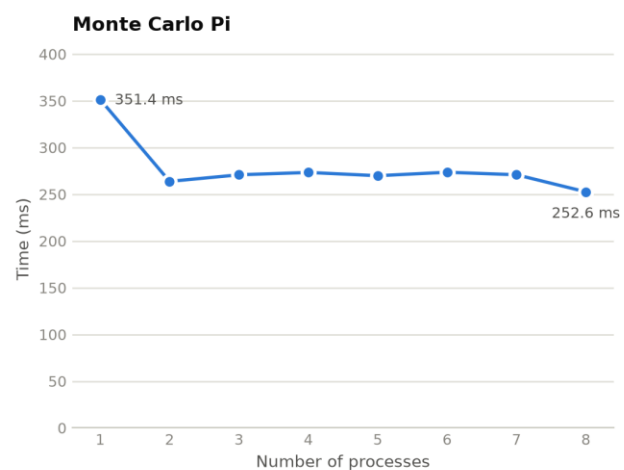
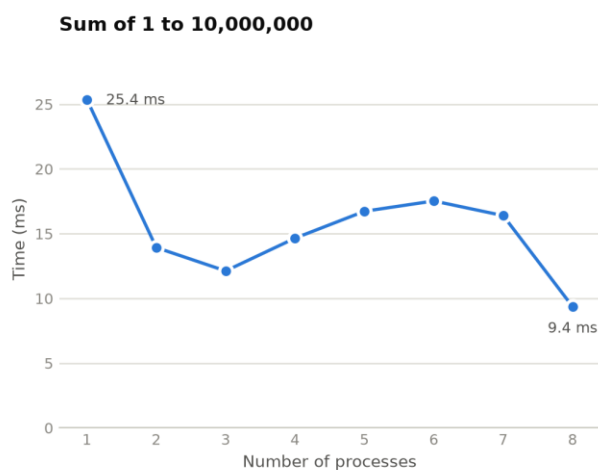
```
[ec2-user@ip-172-31-20-184 lab8]$ nproc
chmod +x run_timings.sh
./run_timings.sh
python3 timing_table.py
2
Run 1 of 10 finished
Run 2 of 10 finished
Run 3 of 10 finished
Run 4 of 10 finished
Run 5 of 10 finished
Run 6 of 10 finished
Run 7 of 10 finished
Run 8 of 10 finished
Run 9 of 10 finished
Run 10 of 10 finished
Saved 320 timings to timings.csv
Median time in seconds (10 runs each)
Processes    pi_anysource    pi_bsend    pi_mpi    sum_mpi
1            0.352585       0.351998    0.351410    0.025367
2            0.264409       0.264343    0.263775    0.013931
3            0.269188       0.276639    0.270823    0.012126
4            0.271872       0.300792    0.273346    0.014655
5            0.276270       0.276176    0.269838    0.016733
6            0.274130       0.278072    0.273563    0.017529
7            0.270494       0.272082    0.270859    0.016399
8            0.261719       0.253722    0.252594    0.009369

Speedup (T1 / Tp)
Processes    pi_anysource    pi_bsend    pi_mpi    sum_mpi
1            1.00x          1.00x       1.00x     1.00x
2            1.33x          1.33x       1.33x     1.82x
3            1.31x          1.27x       1.30x     2.09x
4            1.30x          1.17x       1.29x     1.73x
5            1.28x          1.27x       1.30x     1.52x
6            1.29x          1.27x       1.28x     1.45x
7            1.30x          1.29x       1.30x     1.55x
8            1.35x          1.39x       1.39x     2.71x
[ec2-user@ip-172-31-20-184 lab8]$
```

The lab sheet says Exercise 1 and Exercise 2; the two programs are the sum (Exercise 2) and the Pi calculation (Exercise 3). Both were run with 1 to 8 processes using run_timings.sh. Every run was repeated 10 times and the median time is used, so that one slow run does not change the result. $\text{Speedup} = T_1 / T_p$. The EC2 instance has only 2 vCPUs (`nproc = 2`), so mpirun needs the oversubscribe option and with more than 2 processes the processes have to share the CPUs.

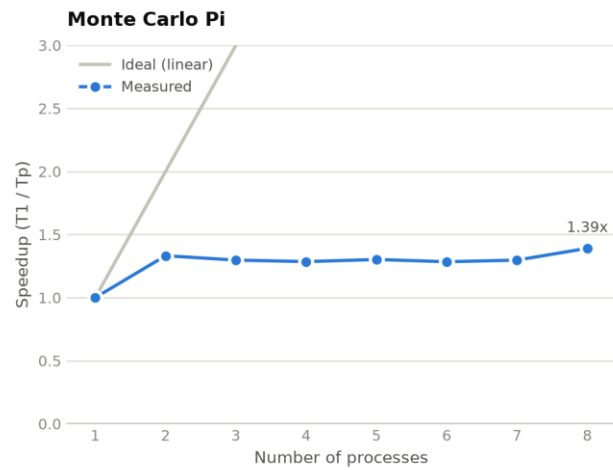
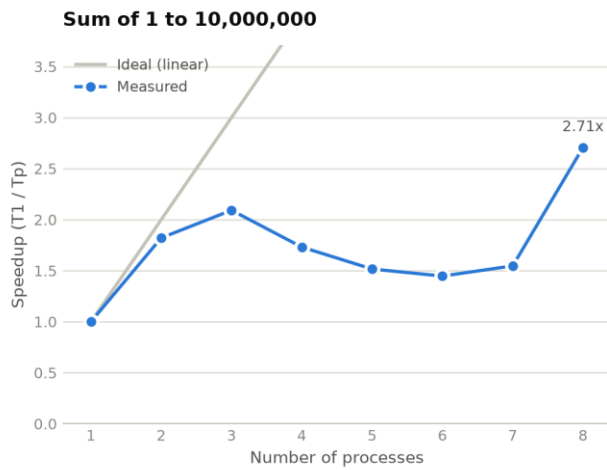
Processes	Sum time (ms)	Sum speedup	Pi time (ms)	Pi speedup
1	25.37	1.00×	351.41	1.00×
2	13.93	1.82×	263.77	1.33×
3	12.13	2.09×	270.82	1.30×
4	14.65	1.73×	273.35	1.29×
5	16.73	1.52×	269.84	1.30×
6	17.53	1.45×	273.56	1.28×
7	16.40	1.55×	270.86	1.30×
8	9.37	2.71×	252.59	1.39×

1. Plot a graph that shows Time vs Number of Processors



The time drops when the second process is added (sum: 25.37 ms → 13.93 ms, Pi: 351.41 ms → 263.77 ms). After that the Pi time stays almost the same from 2 to 7 processes (264 to 274 ms) and is only slightly lower with 8 (252.59 ms). There are only 2 vCPUs, so with more than 2 processes the processes take turns on the CPUs and the total amount of work is the same. The sum time goes up and down between 9.4 ms and 17.5 ms because the whole run is very short.

2. Plot a graph that shows Speedup



The Pi program reaches 1.33× with 2 processes and stays at about 1.3× up to 8 processes (1.39×), far below the ideal line. More processes than vCPUs cannot give more speedup. It is below 2× even with 2 processes, most likely because the 2 vCPUs are two hardware threads of one physical core and not two full cores.

The sum reaches 1.82× with 2 processes. Its values above 2 processes (2.09× at 3, 1.45× at 6, 2.71× at 8) are not a real speedup on 2 vCPUs. The run takes only 10 to 25 ms and the time is measured on root from the moment root leaves MPI_Barrier, so when the processes are taking turns on the CPUs the other processes can already have done part of their loop before the timer on root starts.